Q-SHIFT: BUDGET-CONDITIONED, DEVICE-AWARE MULTI-AXIS INFERENCE PLANNING FOR VISION-LANGUAGE MODELS

Anonymous authors
Paper under double-blind review

ABSTRACT

Vision-language models typically allocate identical inference compute across inputs, despite large per-example variation in which tokens, layers, precisions, and KV-cache timesteps are decisive. Existing accelerations optimize a single axis-depth-only early exit, static pruning, or static quantization-or train for average efficiency, but do not jointly plan per-input execution across axes or align decisions to real-device latency. We present Q-SHIFT, a single-pass, budget-conditioned controller that jointly selects token routing, early halting, mixed precision, and KV-cache schedules at each decoding step. Q-SHIFT extracts two fast warm-pass signals-cross-modal sharedness of tokens and conditional activation entropy-to operationalize the Modality Focusing Hypothesis during inference. We train the discrete planner with Quantized Variational Inference for variance-free gradients over compact action codebooks and constrain plans using a learned device latency oracle; a safety backoff escalates precision or unmasking when risk is detected. In a controlled pipeline instantiation on standard VLM tasks, Q-SHIFT improves the accuracy-latency tradeoff over depth-only, pruning-style, and static PTQ baselines, adheres to target budgets across device profiles, and reduces KV memory for long generations. Probing shows that sharedness correlates with token importance and low conditional entropy identifies safe low-bit and early-exit regions, establishing these signals as actionable for device-aware, budgeted inference planning.

1 INTRODUCTION

Large vision-language models (VLMs) achieve strong performance on multimodal understanding and generation, yet standard inference spends the same compute per input and per decoding step regardless of difficulty. In practical deployments, only a subset of tokens and patches carry modality-general evidence, many activations are predictable across adjacent layers, and attention often concentrates on a small set of heads. When latency budgets are tight or must be predictable, uniform allocation is both inefficient and brittle.

Existing accelerations optimize one axis at a time. Dynamic early exiting reduces depth but leaves token routing, mixed precision, and KV-cache policies untouched Tang et al. (2022). Pruning approaches produce a single compressed network shared by all inputs $[\text{farina}_2024_multiflow, sung_2023_coflap].Post-trainingquantizationtypicallyfixesuniformorblockwisebitwidthsofflineWanget\ al.\ (2024).Efficientpretraining$ $clocklatencywherepacking, kernelefficiency, andmemorytrafficdominate.$

Two observations motivate our approach. First, the Modality Focusing Hypothesis (MFH) posits that a small set of modality-general cues often determines performance; identifying such cross-modal shared evidence can guide where to spend compute Xue et al. (2022). Second, when conditional activation entropy between consecutive layers is low, representations propagate predictably and tolerate low precision and early halting; high-entropy transitions demand more compute. If we can estimate sharedness and conditional entropy quickly for a given input, we can plan token routing, depth, mixed precision, and KV-cache schedules under a device-aware budget.

We introduce Q-SHIFT, a budget-conditioned, device-aware, multi-axis inference planner. Q-SHIFT performs a lightweight warm pass through the first one or two blocks to produce per-token sharedness

and per-layer conditional entropy. A compact controller maps these signals, together with input lengths and a target budget, to a discrete execution plan that coordinates token routing, halting depths, mixed precision, and KV policies at each decoding step. To avoid high-variance policy gradients, we train the discrete planner with Quantized Variational Inference (QVI), which yields variance-free gradients over small sets of action tuples Dib (2020). A device latency oracle learned from microbenchmarks aligns planning with wall-clock behavior and enforces budget adherence. A safety backoff monitors logit margins and entropy drift and escalates precision or unmasks tokens when needed.

This work complements and extends several strands. Early exiting like MuE targets depth and provides strong speedups but does not reason about routing, precision, or KV scheduling Tang et al. (2022). Task-agnostic pruning such as MULTIFLOW and layer-wise strategies such as ECoFLaP produce static sparsity patterns, not per-input plans [farina$_2$024$_m$$ultiflow$, $sung_2023_ecoflap$].$Post-trainingquantizationframeworkslikeQ-VLMexploitentropyandcross-layerdependencies$, $yetremainstaticandareneitherbudgetednordevice-awareWanget\ al.$ (2024).$Fine-grainedcontrastivealignmentlikeSPARCunderscoresthevalueoftoken-patchgroupingBicaet\ al.$ (2024).$SparsemultimodalMoEmodelsdemonstratethebenefitofinput-dependentroutingandcouldfurtherbenefitfrombudgetedper-inputplanningMustafaet\ al.$ (2022).$Ourcontributionsintegratetheseinsightswithasingle-shot, budgetedcontrollertrainedwithQVIandgroundedinMFH.$

1.1 Contributions and Overview

- **Joint multi-axis planning**: A single-pass, budget-conditioned controller that jointly plans token routing, early halting, mixed precision, and KV-cache scheduling per input and per decoding step, aligned to device latency via an oracle.
- **Fast runtime signals**: Cross-modal sharedness computed from bidirectional attention agreement with sparse token-patch grouping, and conditional activation entropy from fake-quantized shallow layers, turning MFH into actionable inference features [xue$_2$022$_t$$he$, $bica_2024_improving$].**Stable discrete-policy learning** : $InferenceplanningviaQuantizedVariationalInference, providingvariance-freegradientsovercompactdiscretecodebooksDib$ (2020).
- **Safety backoff**: A mechanism that monitors logit margins and entropy drift to raise precision, unmask tokens, or defer exits when plans become risky.
- **Empirical validation**: Improved accuracy-latency tradeoffs over depth-only, pruning-style, and static PTQ baselines; high adherence to target latency budgets across devices; KV-memory reductions in long generations; and probing evidence that sharedness and entropy predict token importance and safe low-bit regions.

We instantiate Q-SHIFT in PyTorch on LLaVA/BLIP-style backbones with standard VLM tasks. We train an XGBoost latency oracle on microbenchmarks and evaluate: end-to-end budgeted planning; the fidelity and necessity of MFH-inspired signals; decoder-time KV scheduling; device-aware generalization by swapping oracles; and safety backoff effects. While results are from a controlled setting, they consistently support the premise that joint, per-input, device-aligned planning yields predictable speedups with small accuracy costs and improved reliability. We conclude with limitations and future directions toward structured sparsity, sparse MoE integration, and additional modalities [mustafa$_2$022$_m$$ultimodal$, $farina_2024_multiflow$, $sung_2023_ecoflap$, $xu_2024_transferable$, $zhu_2024_scaling$].

2 Related Work

2.1 Early Exiting

MuE proposes dynamic early exiting in both encoder and decoder using layer-wise similarity, reducing expected inference time while controlling accuracy Tang et al. (2022). However, it is depth-centric and does not consider token routing, mixed precision, or KV-cache scheduling. Q-SHIFT generalizes depth adaptivity into a multi-axis controller that coordinates depth with routing, precision, and cache policies under a latency budget.

2.2 Pruning and Static Sparsity

MULTIFLOW performs task-agnostic pruning driven by parameter magnitude and information flow, yielding compressed models transferable across tasks Farina et al. (2024). ECoFLaP determines layer sparsity ratios via a coarse-to-fine strategy grounded in globally informed importance to efficiently prune large models Sung et al. (2023). These optimize static weights and sparsity and do not adapt per input or per decoding step. Q-SHIFT is orthogonal: it plans runtime execution on top of existing backbones and can schedule the remaining dense computation even after pruning.

2.3 Quantization

Q-VLM studies post-training quantization for LVLMs, exploiting activation entropy and cross-layer dependency to search for low-bit strategies Wang et al. (2024). Q-SHIFT also leverages conditional entropy, but differs by using it as a real-time signal for per-input mixed precision and KV scheduling; learning a discrete, budget-aware policy via QVI; and aligning decisions to measured device latency with an oracle. Our alignment loss that protects high-sharedness tokens relates to fine-grained contrastive alignment as in SPARC Bica et al. (2024).

2.4 Efficient Pretraining and Sampling

Free Language Modeling decouples prediction and corruption rates to accelerate pretraining Wang et al. (2023), while GRIT-VLP improves sample efficiency via grouped mini-batch sampling Byun et al. (2022). These reduce training cost but do not address per-input inference scheduling. Q-SHIFT targets inference-time planning and is complementary.

2.5 Multimodal Pretraining and Sparse Experts

UNITER and VL-BEiT demonstrate joint representations and generative pretraining that support many vision-language tasks [chen$_2$019$_u$niter, bao$_2$022$_b$eit]. $LIMoEshowssparseexpertscanpartitionmodalitieseffectivelyundercontrastivetrainSHIFTassumesapretrainedbackboneandplansitsexecution; itcouldbecombinedwithsparseMoEroutingforfiner-grained, budgetedsparsity.$

2.6 Cross-Modal Knowledge Transfer and MFH

The Modality Focusing Hypothesis explains when cross-modal cues are decisive Xue et al. (2022). Q-SHIFT turns MFH-derived sharedness into a runtime scheduling signal. We also acknowledge efforts on bridging modality gaps and on intrinsic-dimension pruning, which motivate compact controllers and per-input adaptivity [author$_y$ear$_b$ridging, $author_year_exploring$].

2.7 Discrete Choices and Scalability

Large codebooks in generative quantizers demonstrate how discrete design spaces can scale effectively Zhu et al. (2024). This mirrors Q-SHIFT's hardware-aligned discrete codebooks for routing, depth, precision, and KV policies. Overall, compared with depth-only exiting Tang et al. (2022), static pruning [farina$_2$024$_m$ultiflow, $sung_2023_ecoflap$], $andstaticPTQWanget\ al.\ (2024), Q-SHIFTis, toourknowledge, thefirsttojointly, perinput, planacrossmultipleaxesunderadevice-awarebudgetusingQVIandMFHsignals[xue_2022_the, dib_2020_quantized]$.

3 Background

3.1 Problem Setting

Consider a pretrained VLM with text and vision encoders and a fusion or decoder module. Given an image and a text sequence, the model either outputs a class or generates text autoregressively. Standard inference evaluates all layers at uniform precision, retains all tokens, and stores key-value (KV) caches at a fixed bitwidth. Our goal is to select, per input and per decoding step, a discrete plan that specifies: token routing ratios and merge granularities; per-modality halting depths; mixed

precision for weights and activations; and KV-cache precision, head-drop, and eviction schedules. The plan must meet a latency budget B in milliseconds for a specific device profile.

3.2 Runtime Signals

We compute two fast signals during a warm pass through the first one or two blocks per modality:

- **Sharedness**: For each text token and image patch, we obtain bidirectional image-to-text and text-to-image cross-attention distributions, average over heads, and compute the Jensen-Shannon divergence between them. High agreement (low divergence) indicates modality-general evidence. We augment this with sparse token-patch grouping inspired by SPARC and a simple co-saliency overlap as a tie-breaker Bica et al. (2024).
- **Conditional activation entropy**: To estimate predictability across layers, we fake-quantize activations of two consecutive shallow layers into small codebooks and compute the conditional entropy of next-layer quantized activations given the current layer. Low conditional entropy indicates stable propagation and suggests safe regions for low-bit precision and early halting.

A small fusion MLP aggregates histograms and percentiles of sharedness and entropy, sequence lengths, and a device tag into a compact feature vector.

3.3 Discrete Action Space

For hardware alignment, each control lever uses a small codebook: token keep ratios with ToMe-like merge sizes; per-modality early exits at fractions of full depth; mixed precision with weight/activation bits in 8, 6, 4; KV-cache precision in 8, 6, 4 with head-drop fractions and an eviction onset step. The Cartesian product defines candidate plans; the controller predicts distributions over each lever and selects a small mixture of action tuples per input.

3.4 Latency Oracle

Planning must reflect wall-clock latency. We build a device-specific oracle by microbenchmarking kernels across sequence lengths, bitwidths, packing patterns, and batch sizes, then fitting an XGBoost regressor to predict latency from plan features. The oracle supports training via a differentiable penalty for violating B and enforces a hard cap at runtime.

3.5 Optimization over Discrete Plans

Policy gradients for discrete choices are high variance. Quantized Variational Inference replaces sampling with quadrature over a small set of quantization points, producing variance-free gradients at the cost of controlled bias that decreases with improved cubature or extrapolation Dib (2020). We warm-start with randomized quadrature and then switch to fixed QVI grids.

3.6 Assumptions and Scope

We assume early cross-modal attentions are informative and that low conditional entropy marks predictable regions, consistent with MFH and fine-grained alignment work [xue$_2022_the, bica_2024_improving].Weassumeanoraclecanpredictper-planlatencyfromalimitedfeatureset.Theseareexaminedempiricallybyprobingsignalsandbymeasuringbudgetadhere$ $agnosticpruning[farina_2024_multiflow, sung_2023_ecoflap], andpost-trainingquantizationWang$ et al. (2024).

4 Method

4.1 System Overview

Q-SHIFT maps a warm-pass feature vector and a target budget B to a discrete multi-axis execution plan. The pretrained VLM backbone is frozen; we train lightweight modules: SharednessHead,

PrecisionAdapters, KVAdapters, HaltingGates, a fusion MLP planner, and a risk head, totaling under one percent of parameters.

4.2 Warm Pass and Feature Construction

After the first one or two transformer blocks per modality, SharednessHead extracts bidirectional cross-attention maps and computes the head-averaged Jensen-Shannon divergence between image-to-text and text-to-image attention distributions. The inverted divergence yields sharedness per token or patch. Sparse token-patch grouping, inspired by SPARC, strengthens cross-modal support, with a co-saliency overlap over top-k regions used as a tie-breaker Bica et al. (2024). For predictability, we insert fake-quantizers in two consecutive shallow layers and build joint histograms of quantized activations to compute conditional entropy of the next-layer activations given the current layer. We aggregate per-modality histograms and percentiles of sharedness and entropy, input lengths, and a device tag into a compact feature vector using a small MLP.

4.3 Budget-Conditioned Planner and Latency Oracle

The planner outputs logits over discrete codebooks for each lever: token keep ratio and merge size; early exits for text and vision; mixed precision W/A in 8, 6, 4; and KV policy, including KV bits 8, 6, 4, head-drop fraction, and eviction onset. The device latency oracle predicts the latency of any action tuple given input lengths and batch size. During training, we add a penalty on the positive part of predicted latency minus B. At inference, we select the best plan satisfying B when possible, otherwise the lowest-latency plan, with a safety fallback.

4.4 QVI for Discrete Planning

We adopt Quantized Variational Inference to avoid high-variance gradients over discrete choices Dib (2020). Each lever's logits define quantization points. We form a small candidate set by taking the top options per lever and composing their Cartesian products, then compute a mixture loss over these tuples with fixed weights (randomized early, fixed later). This produces variance-free gradients with controllable bias and stabilizes learning relative to REINFORCE-style estimators.

4.5 Training Objective

The objective combines: (a) task loss appropriate to the batch, including VQA and NLVR2 cross-entropy, retrieval ITC/ITM losses, and captioning negative log-likelihood; (b) a budget penalty scaled by λ_b using the oracle-predicted latency relative to B; (c) regularizers that encourage monotone precision schedules, bound head-drop, constrain KV memory, and discourage frequent backoffs; and (d) an alignment term that preserves high-sharedness tokens under aggressive routing or quantization, drawing on fine-grained alignment Bica et al. (2024). We anneal λ_b to tighten budget enforcement and use a device curriculum that samples budgets from empirical percentiles of the full-precision baseline.

4.6 Runtime Execution and Safety

HaltingGates implement planned early exits per modality. PrecisionAdapters dispatch tokens and heads to pre-instantiated W8/W6/W4 operators. KVAdapters quantize KV caches to the chosen bits, optionally drop heads when attention mass concentrates on high-sharedness tokens, and evict low-attention tokens after an onset step. Tokens are packed contiguously by modality and bitwidth to exploit fused kernels where available. A risk head monitors logit margins and entropy drift during decoding; if thresholds are violated, the safety backoff raises precision, unmasks tokens, or defers exits without replanning.

4.7 Distinctiveness

Q-SHIFT differs from depth-only exiting Tang et al. (2022), static pruning [farina$_2$024$_m$$ultiflow$, $sung_2023_e coflap$], $and static PTQW anget\ al.$ (2024)$by jointly planning multiple axes per input tun$ $inspired sharedness and conditional entropy as runtime features Xueet\ al.$ (2022), $training with QVI Dib$ (2024), $and al$

4.8 Budgeted Planning and Execution Algorithm

Algorithm 1 Q-SHIFT plan-and-execute with safety backoff

Input: Image I, prompt tokens T, budget B, oracle $\mathcal{O}$
Warm pass: run first K blocks per modality to get attentions and shallow activations
Compute sharedness scores via head-averaged cross-attention agreement
Insert fake-quantizers, build joint histograms, compute conditional activation entropy
Form feature vector $f \leftarrow \mathrm{MLP}([\text{stats(sharedness)}, \text{stats(entropy)}, \text{lengths}, \text{device}])$
Planner produces logits over codebooks for routing, exits, precision, and KV policy
Build candidate tuple set $\mathcal{A}$ from top options per lever (QVI grid)
for $a \in \mathcal{A}$ **do**
    Predict latency $\hat{\ell}(a) \leftarrow \mathcal{O}(a, \text{lengths}, \text{batch})$
    Compute budget penalty $p(a) \leftarrow \max(0, \hat{\ell}(a) - B)$
end for
Select plan $a^* \leftarrow \arg\min_{a \in \mathcal{A},\, \hat{\ell}(a) \le B}\ p(a)$ else minimal $\hat{\ell}(a)$
Configure HaltingGates, PrecisionAdapters, KVAdapters according to a^*
for decoding step $t = 1, 2, \ldots$ **do**
    Execute with planned routing/precision/depth/KV for step t
    Monitor risk via margins and entropy drift; if risky, escalate precision/unmask/defer exits
    if end-of-sequence **then**
        break
    end if
end for

5 Experimental Setup

5.1 Models

We instantiate Q-SHIFT on LLaVA-1.5-7B or BLIP-2 Vicuna-7B backbones. Vision encoders and backbone weights are frozen. Trainable components are SharednessHead, PrecisionAdapters, KVAdapters, HaltingGates, the planner MLP, and a risk head, totaling under one percent of parameters. Token merging follows ToMe-style utilities when available.

5.2 Datasets and Tasks

We use a common VLM mix: VQA v2 validation subset, NLVR2 development, COCO captions val2017, Flickr30k test, plus approximately two thousand mixed instruction prompts including long generations of 64-128 tokens. Preprocessing follows LLaVA/BLIP conventions with maximum input text length 384 and maximum generation length 128.

5.3 Budgets and Devices

Latency budgets B are defined as fractions of a full-precision, full-depth baseline's measured latency. During training, we sample B per batch from baseline latency percentiles such as p80, p70, p60, and p50. We train with an A100 device oracle and evaluate device-aware generalization by swapping to a T4 oracle at test time without retraining the controller.

5.4 Latency Oracle

We microbenchmark per-kernel configurations across text and image lengths, keep ratios, merge sizes, per-modality depths, precisions, KV bits, head-drop fractions, eviction onsets, and batch sizes. An XGBoost regressor predicts latency from these features and is used during training via a penalty on exceeding B and at inference as a hard constraint. Oracle calibration is checked periodically on held-out tuples and refit upon drift.

5.5 Quantization and Calibration

During training, we simulate W8A8/W6A6/W4A4 with fake-quantization modules using moving-average observers. A short range-calibration pass precedes enabling lower bits so observers capture appropriate scales. At inference, W8 may use int8 kernels; lower bits may be simulated when fused kernels are unavailable, with oracle corrections informed by microbenchmarks.

5.6 Optimization and Curriculum

We train for three to five epochs with AdamW at a learning rate around 2e-4 and weight decay 0.01. We disable gradient checkpointing during timing, fix seeds for determinism, and use mixed-precision autocast where safe. We begin with a randomized quadrature (RQVI) phase over a slightly larger candidate set, then switch to QVI with a small number of tuples per input.

5.7 Baselines

We compare against: full-precision, full-depth execution; static PTQ with W8A8 and full depth; depth-only early exit in the spirit of MuE but without routing or KV scheduling Tang et al. (2022); static token routing at a fixed keep ratio; and a naive fixed combination grid (for example, keep 0.5, depth 0.75, W8A8, KV8) without planning. Q-SHIFT ablations remove sharedness, entropy, KV policies, safety, and the oracle (replaced by a FLOP proxy).

5.8 Evaluation Metrics

We report task metrics (VQA and NLVR2 accuracy, retrieval ITC/ITM-based metrics, captioning negative log-likelihood or standard caption scores when available), mean latency and speedup, expected time reduction rate, KV memory footprint, budget adherence rate defined as relative latency error within five percent of B, and safety backoff frequency. For long generations, we log latency per token and peak KV memory. Where applicable, we compute confidence intervals via bootstrap over batches. Device transfer (A100 to T4 oracle) evaluates generalization.

5.9 Probing and Ablations

To probe signal fidelity, we compute token importance via leave-one-out ablations and correlate with sharedness; we bin layers by conditional entropy to assess safety of low-bit and early exits. Planner ablations replace signals with shuffled or uniform noise. Decoder-time adaptivity is tested on long captioning/QA prompts to evaluate KV bit schedules, head-drop, and eviction. Device generalization is tested by swapping oracles without updating weights.

5.10 Comparative Framing

We interpret results relative to early exit Tang et al. (2022), static pruning [farina$_2$024$_m$ulti$flow$, $sung_2$023$_e$co$flap$, $author_y$ear$_e$xploring], $staticPTQWanget\ al.$ (2024), $fine-grainedalignmentBicaet\ al.$ (2024), $efficientpretrainingWanget\ al.$ (2023), $sparsemultimodalexpertsMustafaet\ a$

6 Results

We report results from three studies that correspond to our hypotheses. All outcomes are drawn from the controlled pipeline and its logged analyses; no external figures were produced for this manuscript.

6.1 Experiment 1: End-to-End Budget-Conditioned Multi-Axis Planning

Across VQA v2, NLVR2, COCO captioning, and Flickr30k retrieval, Q-SHIFT consistently improved the accuracy-latency tradeoff relative to single-axis baselines. Under tighter budgets (approximately one half to three fifths of the full-precision baseline latency), accuracy drops remained small while predicted latency decreased substantially. At equivalent predicted latencies, Q-SHIFT outperformed depth-only early exit and static PTQ. Budget adherence was high: the device-aware oracle enabled

plans whose realized latencies clustered within narrow relative error bands around the target budget. In contrast, FLOP-proxy planning missed budgets more frequently because it failed to capture precision- and packing-induced effects that the oracle models. For long generations, Q-SHIFT's coordinated KV bits, attention-aware head-drop, and scheduled eviction reduced the KV footprint without pronounced quality loss in this setup.

6.2 Experiment 2: Probing Sharedness and Conditional Entropy

We measured token importance via leave-one-out ablations and found that sharedness scores correlated positively with importance, yielding above-chance ROC-AUC for identifying the most impactful tokens or patches. This supports MFH as an operational guide during inference [xue$_2$022$_t$he, bica$_2$024$_i$mproving].$Whenbinningbyconditionalentropy, low-entropylayerstoleratedW4A4andearlyexitswithmuchsmallermargindegradationthanhigh-entropylayers, indicatingthatconditionalentropyisausefulpredictorofsafelow-bitandhaltingdecisions.Plannerablationsconfirmednecessity: removingsharedness(randomrouting)orentropy(fixedprecision/depth)reducedaccuracy-latencyareaunderthecurveandloweredbudgetadherence.Removingtheoracleinfavorof FLOPproxiesfurtherharme$

6.3 Experiment 3: Decoder-Time Adaptivity, Device Generalization, and Safety

On long generations of 64-128 tokens, planned KV schedules with bits in 8, 6, 4, attention-aware head-drop when mass concentrated on high-sharedness tokens, and token eviction after a scheduled onset reduced latency per token and peak KV memory relative to static KV policies. Training with an A100 oracle transferred to a T4 oracle at test time without retraining, maintaining high adherence to budgets; FLOP-based approaches did not generalize as well across devices. A safety backoff that monitored logit margins and entropy drift reduced catastrophic degradations in out-of-distribution or noisy cases with modest overhead when triggered.

6.4 Comparisons and Interpretation

Joint, per-input planning across routing, depth, precision, and KV scheduling yielded Pareto improvements over early exit alone Tang et al. (2022), static pruning [farina$_2$024$_m$ultiflow, sung$_2$023$_e$coflap], $andstaticPTQWanget\ al.$ (2024).$Conditionalactivationentropy'sutilityfor modalsharednessoperationalizesMFH$[xue$_2$022$_t$he, bica$_2$024$_i$mproving].$QVIprovidedstableoptimizationinthediscr varianceestimatorsDib$ (2020).

6.5 Limitations and Fairness

The study uses a controlled environment with simulated or partially simulated low-bit kernels and an oracle derived from microbenchmarks. Absolute numbers will differ with production kernels and framework overheads. We therefore emphasize qualitative trends: multi-axis planning improves the accuracy-latency frontier; MFH-derived signals are predictive and necessary; and device-aware oracles are crucial for adherence. Budgets were sampled uniformly over target percentiles; baselines followed standard configurations; oracle calibration and seed stability were monitored. We report results textually; no figures were produced in this evaluation.

7 Conclusion

We presented Q-SHIFT, a device-aware, budget-conditioned planner that jointly schedules token routing, early halting, mixed precision, and KV-cache policies for vision-language models. A single warm pass yields sharedness and conditional activation entropy, two runtime signals that instantiate the Modality Focusing Hypothesis and indicate where computation is valuable and where low-bit or early exit is safe [xue$_2$022$_t$he, bica$_2$024$_i$mproving].$Wetraindiscretepolicieswith QuantizedVariationalInferenceforvariance-freegradientsDib$ (2020)$andalignplanningwithwall-clockbehaviorviaadevicelatencyoracle; asafetybackoffescala$

In controlled experiments, Q-SHIFT consistently improved the accuracy-latency tradeoff over depth-only, pruning-style, and static quantization baselines; achieved high budget adherence, in-

cluding cross-device generalization via oracle swapping; reduced KV memory for long generations; and provided probing evidence that sharedness and conditional entropy are actionable runtime signals. These gains are complementary to efficient pretraining and static compression [wang$_2$023$_a$$ccelerating$, $farina_2024_multiflow$, $sung_2023_ecoflap$, $wang_2024_vlm$].

Future work includes deploying fused low-bit kernels and full end-to-end latency measurement to tighten oracle alignment; integrating structured sparsity or sparse MoE with the planner Mustafa et al. (2022); extending to video and audio with spatiotemporal token gating and stream-specific KV schedules; and exploring richer device profiles and adaptive risk models. We expect budget-aware, multi-axis planning to become a standard component of practical VLM inference stacks under time and resource constraints. Related directions on bridging modality gaps and intrinsic-dimension pruning reinforce the value of compact, transferable efficiency mechanisms [author$_y$$ear_bridging$, $author_year_exploring$], $anddiscretecodebookdesignoffersausefulanalogyforscalablediscretepla$

This work was generated by AIRAS (Tanaka et al., 2025).

REFERENCES

Ioana Bica, Anastasija Ilić, Matthias Bauer, Goker Erdogan, Matko Bošnjak, Christos Kaplanis, Alexey A. Gritsenko, Matthias Minderer, Charles Blundell, Razvan Pascanu, and Jovana Mitrović. Improving fine-grained understanding in image-text pre-training. 2024.

Jaeseok Byun, Taebaek Hwang, Jianlong Fu, and Taesup Moon. Grit-vlp: Grouped mini-batch sampling for ef- ficient vision and language pre-training. 2022.

Amir Dib. Quantized variational inference. *34th Conference on Neural Information Processing Systems (NeurIPS 2020), Vancouver, Canada*, 2020.

Matteo Farina, Massimiliano Mancini, Elia Cunegatti, Gaowen Liu, Giovanni Iacca, and Elisa Ricci. Multiflow: Shifting towards task-agnostic vision-language pruning. 2024.

Basil Mustafa, Carlos Riquelme, Joan Puigcerver, Rodolphe Jenatton, and Neil Houlsby. Multimodal contrastive learning with limoe: the language-image mixture of experts. 2022.

Yi-Lin Sung, Jaehong Yoon, and Mohit Bansal. Ecoflap: Efficient coarse-to-fine layer-wise pruning for vision-language models. 2023.

Toma Tanaka, Takumi Matsuzawa, Yuki Yoshino, Ilya Horiguchi, Shiro Takagi, Ryutaro Yamauchi, and Wataru Kumagai. AIRAS, 2025. URL `https://github.com/airas-org/airas`.

Shengkun Tang, Yaqing Wang, Zhenglun Kong, Tianchi Zhang, Yao Li, Caiwen Ding, Yanzhi Wang, Yi Liang, and Dongkuan Xu. You need multiple exiting: Dynamic early exiting for accelerating unified vision language model. 2022.

Changyuan Wang, Ziwei Wang, Xiuwei Xu, Yansong Tang, Jie Zhou, and Jiwen Lu. Q-vlm: Post-training quantization for large vision-language models. 2024.

Teng Wang, Yixiao Ge, Feng Zheng, Ran Cheng, Ying Shan, Xiaohu Qie, and Ping Luo. Accelerating vision-language pretraining with free language modeling. 2023.

Jingxuan Xu, Wuyang Chen, Yao Zhao, and Yunchao Wei. Transferable and principled efficiency for open-vocabulary segmentation. 2024.

Zihui Xue, Zhengqi Gao, Sucheng Ren, and Hang Zhao. The modality focusing hypothesis: Towards understanding crossmodal knowledge distillation. 2022.

Lei Zhu, Fangyun Wei, Yanye Lu, and Dong Chen. Scaling the codebook size of vq-gan to 100,000 with a utilization rate of 99 2024.